

Chapter 10: Vehicle Routing Problems with Profits

Vehicle Routing: Problems, Methods, and Applications (2nd ed., 2014)

Archetti, Speranza & Vigo

Lecture notes prepared for self-study

2024

Outline I

- 1 Introduction
- 2 Single-Vehicle Case
 - Orienteering Problem (OP)
 - Prize-Collecting TSP (PCTSP)
 - Profitable Tour Problem (PTP / DCTSP)
- 3 Multiple-Vehicle Case
 - Team Orienteering Problem (TOP)
 - Capacitated Team Orienteering Problem (CTOP)
- 4 Variants
 - Time-Window Variants
 - Clustered Variants
 - Multi-Period Variants
- 5 Solution Methods: Code Examples
- 6 Conclusions and Future Research Directions
- 7 Summary of Key Formulas
- 8 Bibliography

What Makes VRPPs Different? I

In classical Vehicle Routing Problems (VRPs), the set of customers to serve is *given in advance* and every customer must be visited. The Vehicle Routing Problems with Profits (VRPPs) relax this assumption in a fundamental way: **visiting a customer is optional**.

Each customer i carries a **profit** p_i — a numerical value that reflects how attractive it is to visit that customer. This profit could represent revenue collected, points scored, or any benefit obtained from the visit.

Because customers need not all be served, the planner must make two intertwined decisions simultaneously:

- 1 **Selection:** which customers to include in the routes, and
- 2 **Sequencing:** in what order to visit the chosen customers.

A route (or a set of routes) starts and ends at a *depot* and can be measured both in terms of **cost** (distance, time, fuel) and in terms of **profit** collected. The defining characteristic of VRPPs is that the difference between profit and cost can be maximised, or one of the two can be optimised while the other is kept within a bound.

What Makes VRPPs Different? II

Three ways to combine profit and cost

- 1 **Maximise profit** subject to a *cost bound* (e.g. a time limit $T_{\max}$): collect as much profit as possible without exceeding the budget.
- 2 **Minimise cost** subject to a *profit threshold* $P_{\min}$: travel as little as possible while still collecting at least $P_{\min}$ profit.
- 3 **Maximise (profit — cost)**: find the route for which the net benefit is greatest, with no hard constraints on either.

Real-World Applications I

VRPPs arise in many practical settings. In each case the key idea is that it is not always worth visiting every location — visiting a low-profit customer far from the depot may cost more than the benefit gained.

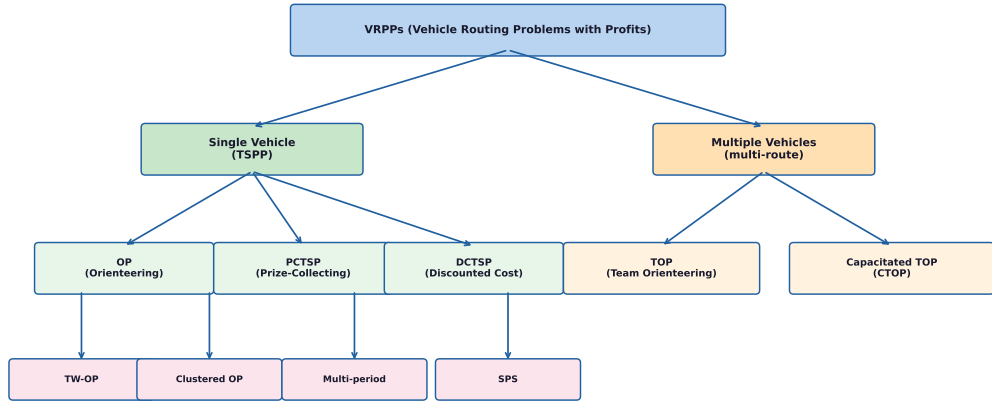
- **Steel rolling mill scheduling.** A rolling mill can produce items for some customers during a given shift. Not all orders can be fulfilled; the scheduler selects which orders to accept to maximise throughput. (Balas, 1989; 1993)
- **Tourist trip design.** Given a fixed vacation time, a tourist visits a subset of attractions. Each attraction has a “score” (profit). The goal is to plan the itinerary that maximises total score within the time budget. (Vansteenwegen & Van Oudheusden, 2007)
- **Auditor routing.** A limited number of auditors must visit suppliers. The profit is the amount of recovered claims. Not all suppliers can be audited; the scheduler selects the most profitable ones. (Ilhan, Iravani & Daskin, 2008)
- **Sports recruiting.** A college recruiter has limited travel days to visit high schools and recruit athletes. Each school has an estimated recruitment value (profit). (Butt & Cavalier, 1994)
- **Salesperson planning.** A salesperson cannot call on all clients each week; she selects those offering the highest expected commission. (Ramesh & Brown, 1991)

- **Oil tanker routing.** An oil company routes tankers to serve ships at sea. Demand at each location is known; only a subset of ships can be refuelled per voyage. (Golden, Levy & Vohra, 1987)
- **Home-care delivery.** A company delivers home-care products. Not all customers need delivery every day; a maximum distance is imposed on each route. (Savelsbergh & Sol, 1995)
- **Unmanned aerial vehicle (UAV) mission planning.** A UAV has limited battery. It visits a subset of targets to collect information, maximising total intelligence value collected.

The VRPP Family: Taxonomy I

The VRPP Family: Taxonomy II

Taxonomy of Vehicle Routing Problems with Profits (VRPPs)



Variants of OP

The VRPP Family: Taxonomy III

Figure: Taxonomy of Vehicle Routing Problems with Profits. Single-vehicle variants form the *Travelling Salesman Problem with Profits (TSPP)* family. Multi-vehicle variants form the *Team Orienteering Problem (TOP)* family.

The taxonomy splits naturally along two dimensions: (1) single vehicle vs. multiple vehicles, and (2) the type of objective (maximise profit, minimise cost, or maximise net profit). Many real-world extensions add further structure: time windows, clustered customers, multi-period planning, or stochastic profits.

Summary Table of VRPP Variants I

The following table summarises the main problem classes studied in this chapter. Column *Objective* shows what is optimised; column *Constraint* shows the binding restriction that makes the problem non-trivial.

Problem	Objective	Key Constraint	Vehicles
OP (Orienteering)	Max profit	Travel time $\leq T_{\max}$	1
PCTSP (Prize-Collecting TSP)	Min cost + penalties	Min profit $\geq P_{\min}$	1
DCTSP (Discounted-Cost TSP)	Max profit – cost	None	1
TOP (Team Orienteering)	Max total profit	Each route $\leq T_{\max}$	m
CTOP (Capacitated TOP)	Max total profit	Demand $\leq Q$ per route	m
TW-OP / TW-TOP	Max profit	Time windows $[a_i, b_i]$	$1/m$
Clustered OP	Max profit	Visit whole cluster or none	1
Multi-period TOP	Max total profit	Period planning horizon	m

In the table, $T_{\max}$ (T-max) is the maximum allowed travel time or distance per route; $P_{\min}$ (P-min) is a minimum profit threshold; Q (Q) is the vehicle capacity; m is the number of vehicles; and $[a_i, b_i]$ (a-sub-i to b-sub-i) is the time window for customer i — the interval during which the vehicle must arrive.

Graph Notation and Problem Setup I

All VRPP variants are defined on a **directed graph** (or equivalently an undirected graph for symmetric costs). We define the notation once here and reuse it throughout.

Graph Notation and Problem Setup II

Graph Notation

- $G = (V, A)$: a complete directed graph where V is the set of nodes and A is the set of arcs (directed edges).
- $V = \{0, 1, 2, \dots, n\}$: node 0 is the *depot* (starting and ending point); nodes $1, \dots, n$ are the *customers* (potential stops).
- $p_i \geq 0$: profit associated with customer $i \in V \setminus \{0\}$. This is the reward collected when customer i is visited.
- $c_{ij} \geq 0$: travel cost (distance or time) from node i to node j . In most instances, c_{ij} is the Euclidean distance between two points in the plane.
- $x_{ij} \in \{0, 1\}$: binary decision variable. $x_{ij} = 1$ if the vehicle travels directly from node i to node j ; otherwise $x_{ij} = 0$.
- $y_i \in \{0, 1\}$: binary decision variable. $y_i = 1$ if customer i is visited; otherwise $y_i = 0$.

The subscript notation x_{ij} is read “x sub i j”. Think of x_{ij} as a switch: it is 1 (on) if the route uses the road segment from i to j , and 0 (off) otherwise.

The Orienteering Problem (OP): Intuition I

The Orienteering Problem (OP) takes its name from the outdoor sport of orienteering, where competitors collect checkpoints within a fixed time. The mathematical problem asks the same question: **which checkpoints (customers) can I visit within a given time limit, and in what order, so as to collect the most points (profit)?**

The OP was first introduced by Tsiligirides (1984) and later formally defined by Golden, Levy, and Vohra (1987). It is a single-vehicle problem: one vehicle (or person) starts at the depot, visits a subset of customers, and returns to the depot, all within a maximum travel time $T_{\max}$ (T-max).

The Orienteering Problem (OP): Intuition II

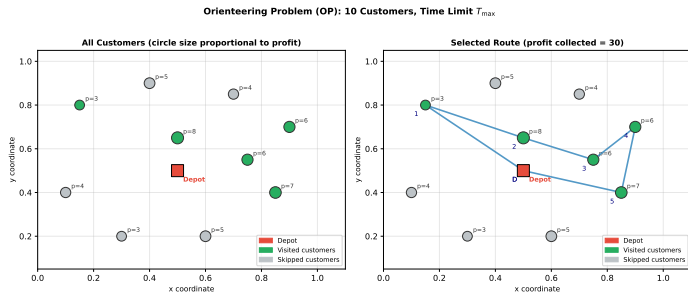


Figure: Left: all 10 customers, circle size proportional to profit. Right: the selected route (blue path) visits 5 customers and collects profit equal to their sum. The remaining customers are skipped because including them would violate the time limit.

OP Mathematical Formulation I

We now state the OP as an integer program. Every constraint is explained in full.

OP Mathematical Formulation II

Orienteering Problem (OP) – Integer Program

Maximise

$$\sum_{i \in V} p_i y_i \quad (\text{OP.obj})$$

Subject to

$$\sum_{j \in V} x_{0j} = 1 \quad (\text{OP.1})$$

$$\sum_{j \in V} x_{j0} = 1 \quad (\text{OP.2})$$

$$\sum_{j \in V} x_{ij} = y_i \quad \forall i \in V \setminus \{0\} \quad (\text{OP.3})$$

$$\sum_{j \in V} x_{ji} = y_i \quad \forall i \in V \setminus \{0\} \quad (\text{OP.4})$$

$$\sum_{(i,j) \in A} c_{ij} x_{ij} \leq T_{\max} \quad (\text{OP.5})$$

OP: A Worked Numerical Example I

We illustrate the OP with 5 customers and a small time limit. Customers are located in the plane; the depot is at the origin $(0, 0)$.

Instance

Customer	x	y	Profit p_i
Depot (0)	0	0	–
1	1	2	5
2	3	1	8
3	4	3	6
4	2	4	4
5	5	2	9

Time limit: $T_{\max} = 12$. Travel cost c_{ij} is Euclidean distance (rounded to 1 decimal place).

Relevant distances:

OP: A Worked Numerical Example II

- $c_{01} = \sqrt{1^2 + 2^2} \approx 2.2$, $c_{02} = \sqrt{3^2 + 1^2} \approx 3.2$, $c_{05} = \sqrt{5^2 + 2^2} \approx 5.4$
- $c_{12} = \sqrt{(3-1)^2 + (1-2)^2} \approx 2.2$, $c_{25} = \sqrt{(5-3)^2 + (2-1)^2} \approx 2.2$, $c_{50} = 5.4$

Route Candidate A: Depot $\rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow$ Depot.

Cost = $2.2 + 2.2 + 2.2 + 5.4 = 12.0 \leq T_{\max}$. Profit = $5 + 8 + 9 = 22$.

Route Candidate B: Depot $\rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow$ Depot.

Cost = $3.2 + \sqrt{(4-3)^2 + (3-1)^2} + \sqrt{(5-4)^2 + (2-3)^2} + 5.4 \approx 3.2 + 2.2 + 1.4 + 5.4 = 12.2 > T_{\max}$.

Infeasible.

Conclusion: Route A is feasible and collects profit 22. Adding customer 3 or 4 pushes cost above 12. Route A is optimal for this instance.

Key Insight

With only 5 customers, checking all $2^5 = 32$ subsets is feasible by hand. For $n = 100$ customers, brute force requires checking 2^{100} subsets — impossible without intelligent algorithms. This is why the OP is NP-hard and requires sophisticated methods.

OP: Exact Algorithms I

The OP is NP-hard (non-deterministic polynomial time hard), meaning no known algorithm solves it optimally in time that grows polynomially with n (the number of customers). Despite this, exact algorithms can solve instances of moderate size.

Branch and Bound (B&B). Tsiligris (1984) first proposed the OP and solved it using B&B. The algorithm builds a tree of partial solutions. At each node it computes an *upper bound* on the best possible profit achievable from that partial solution. If the upper bound is less than the best complete solution found so far, the branch is pruned (discarded). Laporte and Martello (1990) developed stronger upper bounds based on a relaxation of the time constraint, reducing the tree size.

Dynamic Programming (DP). Tsiligris (1984) also proposed a DP approach. The state is the set of customers already visited and the current node; the value function stores the maximum profit achievable. DP is exact but requires memory exponential in n (the state space grows as $2^n \times n$), so it is practical only for $n \lesssim 20$.

Branch and Cut (B&C). Fischetti, Salazar-González, and Toth (1998) developed a branch-and-cut approach using subtour elimination constraints added dynamically. This method can solve instances with up to about 100 customers to optimality.

Integer Linear Programming (ILP) / Branch and Price. For larger instances, column generation combined with B&B (branch-and-price) has been used. Each column in the master problem corresponds to a complete route; the pricing subproblem finds the most profitable route to add.

OP: Approximation Results I

Because the OP is NP-hard, researchers have studied how well polynomial-time algorithms can approximate the optimal solution.

Approximation Hardness

The OP cannot be approximated within any constant factor in polynomial time unless $P = NP$ (where P and NP are complexity classes). Intuitively, this is because the subset-selection and routing decisions are tightly coupled.

Blum et al. (2003) proposed the first polynomial-time approximation scheme (PTAS) for a special case where the time limit is large enough to visit most customers. Chekuri et al. (2012) showed that the OP can be approximated within a factor of $(1 + 1/\varepsilon)$ (1 plus 1-over-epsilon) for any $\varepsilon > 0$ (epsilon greater than zero) when using a quasi-polynomial algorithm (running in time $n^{O(\log n)}$).

OP: Approximation Results II

Practical Implication

For the instances sizes arising in practice (typically $n \leq 300$), metaheuristics (Tabu Search, Simulated Annealing, ILS) find solutions within 1–2% of optimal in seconds, making them the method of choice.

OP: Heuristic Algorithms – Greedy Construction I

Greedy construction heuristics build a solution step by step, adding one customer at a time without backtracking. They are fast but may miss the global optimum.

The Profit-per-Cost Ratio Heuristic. The simplest greedy approach ranks customers by their profit-to-distance ratio and inserts them greedily:

- 1 Compute the ratio $r_i = p_i/c_{0i}$ (r_i = profit of customer i divided by distance from depot to i) for each customer.
- 2 Sort customers by r_i in decreasing order.
- 3 Starting from an empty route (depot $\rightarrow$ depot), insert the next customer from the sorted list into the cheapest position in the current route, as long as the time limit is not violated.
- 4 Repeat until no more customers can be inserted without exceeding $T_{\max}$.

Worked example (5 customers from the previous slide):

- Ratios: $r_1 = 5/2.2 \approx 2.3$, $r_2 = 8/3.2 = 2.5$, $r_3 = 6/5.1 \approx 1.2$, $r_4 = 4/4.5 \approx 0.9$, $r_5 = 9/5.4 \approx 1.7$.
- Sorted order: 2, 1, 5, 3, 4.

OP: Heuristic Algorithms – Greedy Construction II

- Insert customer 2: route = $D \rightarrow 2 \rightarrow D$, cost = $3.2 + 3.2 = 6.4$.
- Insert customer 1: route = $D \rightarrow 1 \rightarrow 2 \rightarrow D$, cost = $2.2 + 2.2 + 3.2 = 7.6$.
- Insert customer 5: route = $D \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow D$, cost = $2.2 + 2.2 + 2.2 + 5.4 = 12.0 \leq 12$. ✓
- Insert customer 3: would cost > 12 . Stop.
- Final profit = $5 + 8 + 9 = 22$. (Optimal for this instance.)

OP: Heuristic Algorithms – GRASP and ILS I

More sophisticated heuristics combine construction with local search improvement.

GRASP (Greedy Randomised Adaptive Search Procedure). GRASP was applied to the OP by Fischetti, Salazar-González, and Toth (1998). Each iteration has two phases:

- 1 **Construction phase:** at each step, instead of always inserting the single best customer, build a *Restricted Candidate List (RCL)* of the top k candidates by profit/cost ratio, then select one at random from the RCL. This introduces controlled randomness.
- 2 **Local search phase:** improve the constructed solution by applying 2-opt or 3-opt exchanges (removing 2 or 3 edges and reconnecting in a better order) and swap moves (exchanging a visited customer with an unvisited one if it improves profit).

The best solution across all iterations is retained. GRASP is easy to parallelise because iterations are independent.

ILS (Iterated Local Search). Vansteenwegen, Souffriau, and Van Oudheusden (2009) developed a very effective ILS for the OP and TOP. ILS alternates between:

- a **perturbation step** that randomly removes some customers and re-inserts others (escaping local optima), and

OP: Heuristic Algorithms – GRASP and ILS II

- a **local search step** that applies 2-opt, Or-opt, and customer swap moves until no improvement is found.

ILS achieved near-optimal results on all standard OP benchmarks in fractions of a second.

Prize-Collecting TSP (PCTSP) I

The Prize-Collecting Travelling Salesman Problem (PCTSP) was introduced by Balas (1989) to model steel rolling mill scheduling. It differs from the OP in both the objective and the constraint structure.

In the PCTSP:

- Each customer i has a **profit** p_i (reward for visiting) and a **penalty** π_i (π_i sub i , pronounced “pie sub i ”) that is paid if customer i is *not* visited.
- The objective is to **minimise total travel cost plus total penalties** for unvisited customers.
- There is a **minimum profit constraint**: the total profit collected must be at least $P_{\min}$ (P-min).

Prize-Collecting TSP (PCTSP) II

PCTSP – Integer Program

Minimise

$$\sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{i \in V \setminus \{0\}} \pi_i (1 - y_i) \quad (\text{PCTSP.obj})$$

Subject to

$$\sum_{i \in V \setminus \{0\}} p_i y_i \geq P_{\min} \quad (\text{PCTSP.profit})$$

plus the same flow conservation constraints as the OP.

Prize-Collecting TSP (PCTSP) III

Explanation of the PCTSP Objective

The objective function (PCTSP.obj) has two parts:

- ① $\sum_{(i,j)} c_{ij}x_{ij}$: total travel cost for all arcs used. Here c_{ij} (c sub i j) is the cost of traveling from node i to node j , and x_{ij} (x sub i j) is 1 if that arc is used.
- ② $\sum_i \pi_i(1 - y_i)$: total penalty for unvisited customers. Here π_i (π sub i) is the penalty for skipping customer i , and $(1 - y_i)$ equals 1 if customer i is skipped.

The minimisation balances these two costs: travelling further reduces penalties but increases route cost.

PCTSP vs OP

In the OP, the time limit is a *hard constraint* and the objective is to maximise profit. In the PCTSP, the minimum profit is a *hard constraint* and the objective is to minimise cost plus penalties. Both are equivalent in the limit (one can be converted to the other by parameterising $T_{\max}$ or $P_{\min}$), but they model different real-world priorities.

Prize-Collecting TSP (PCTSP) IV

Exact algorithms for the PCTSP. Fischetti and Toth (1988) introduced the first exact algorithm. Bienstock, Goemans, Simchi-Levi, and Williamson (1993) showed that the PCTSP admits a 2-approximation (a polynomial algorithm guaranteed to find a solution within a factor of 2 of optimal).

Profitable Tour Problem (PTP) and DCTSP I

The Profitable Tour Problem (PTP), also called the Discounted-Cost TSP (DCTSP), takes yet another approach to balancing profit and cost.

In the PTP:

- There is **no hard constraint** on either total travel cost or minimum profit.
- Each customer i has a profit p_i for visiting and a cost c_{ij} for each arc.
- The objective is to **maximise net profit**: total collected profit minus total travel cost.

PTP – Objective

Maximise

$$\sum_{i \in V \setminus \{0\}} p_i y_i - \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (\text{PTP.obj})$$

Subject to the same flow conservation and integrality constraints as the OP (but without the time limit constraint OP.5).

Profitable Tour Problem (PTP) and DCTSP II

Here $\sum_i p_i y_i$ (Sigma p sub i y sub i) is the total profit from visited customers, and $\sum_{(i,j)} c_{ij} x_{ij}$ (Sigma c sub i j times x sub i j) is the total travel cost. The difference is maximised.

Key behavioural observation. In the PTP, a customer i is worth visiting only if its profit p_i exceeds the detour cost of including it. Formally, customer i is worth adding to a route that currently goes from node a to node b if $p_i > c_{ai} + c_{ib} - c_{ab}$ (the profit exceeds the savings lost from the detour).

Exact algorithms. Del Pia and Filippi (2006) and Archetti, Hertz, and Speranza (2007) studied exact and heuristic methods. Archetti et al. proposed a branch-and-cut approach that adds subtour elimination and optimality cuts dynamically.

From Single to Multiple Vehicles I

When a single vehicle cannot collect enough profit within the time limit, deploying a **fleet of vehicles** becomes necessary. Multiple vehicles depart from the same depot, each follows an independent route within the time limit $T_{\max}$, and together they aim to collect the maximum total profit.

The transition from the OP to the Team Orienteering Problem (TOP) is analogous to the transition from the TSP to the VRP: we add more vehicles and ask how to partition the customers among them optimally.

New notation for multi-vehicle problems

- m : number of vehicles in the fleet.
- $k \in \{1, \dots, m\}$: index for vehicles.
- $x_{ijk} \in \{0, 1\}$: equals 1 if vehicle k travels directly from node i to node j .
- $y_{ik} \in \{0, 1\}$: equals 1 if vehicle k visits customer i .
- $T_{\max}$: maximum travel time (or distance) allowed for each individual vehicle's route.

A customer can be visited by **at most one vehicle**. If customer i is visited, exactly one vehicle serves it. This prevents double-counting profits.

Team Orienteering Problem (TOP): Definition I

The Team Orienteering Problem (TOP) was introduced by Chao, Golden, and Wasil (1996). It generalises the OP to m vehicles: **find m routes, each starting and ending at the depot and not exceeding $T_{\max}$, so that the total collected profit is maximised.**

Team Orienteering Problem (TOP): Definition II

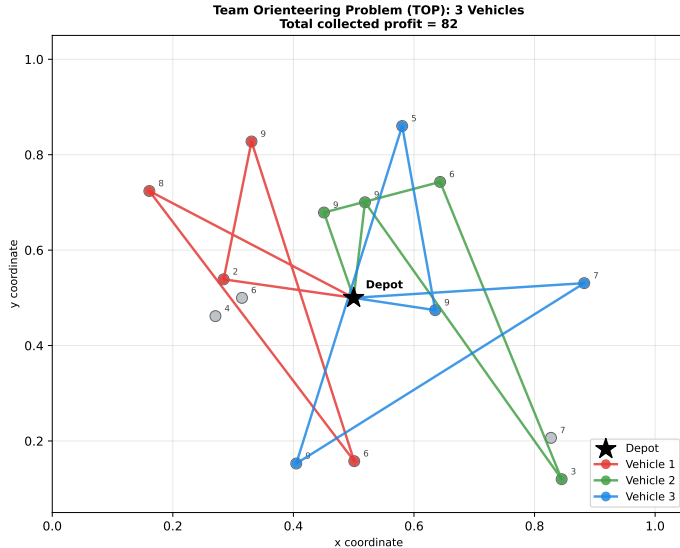


Figure: TOP with 3 vehicles and 15 customers. Each coloured path is one vehicle's route. Not all customers are

TOP: Mathematical Formulation I

We present the full integer programming formulation for the directed TOP.

TOP: Mathematical Formulation II

Team Orienteering Problem – Integer Program

Maximise

$$\sum_{k=1}^m \sum_{i \in V} p_i y_{ik} \quad (\text{TOP.obj})$$

Subject to

$$\sum_{k=1}^m \sum_{j \in V} x_{0jk} = m \quad (\text{TOP.1})$$

$$\sum_{j \in V} x_{0jk} = 1 \quad \forall k \quad (\text{TOP.2})$$

$$\sum_{j \in V} x_{j0k} = 1 \quad \forall k \quad (\text{TOP.3})$$

$$\sum_{j \in V} x_{ijk} = y_{ik} \quad \forall i \in V \setminus \{0\}, k \quad (\text{TOP.4})$$

$$\sum_{j \in V} x_{jik} = y_{ik} \quad \forall i \in V \setminus \{0\}, k \quad (\text{TOP.5})$$

TOP: Heuristic Algorithms I

Because the TOP is NP-hard, heuristic methods dominate the practical literature. The following approaches have been most successful.

1. **Greedy Algorithm (Chao, Golden, Wasil 1996).** The first heuristic for the TOP proceeds greedily:
 - 1 For each vehicle k in turn, select the unvisited customer with the best profit-per-distance ratio that can be inserted without violating $T_{\max}$.
 - 2 Insert it into the cheapest position in vehicle k 's current route.
 - 3 Repeat until no more insertions are feasible.

This is fast ($O(n^2m)$ time) but may be far from optimal because it does not reconsider earlier decisions.

2. **Simulated Annealing (SA) (Chiang and Wang, 1996).** Simulated Annealing mimics the physical process of cooling a metal. A hot metal can move to a higher-energy state (worse solution) with non-zero probability; as it cools, the probability of accepting worse solutions decreases. In the TOP context:
 - The “temperature” T (T) starts high and is reduced by a cooling factor α (alpha) at each iteration:
$$T \leftarrow \alpha \cdot T.$$

TOP: Heuristic Algorithms II

- At each step, a *neighbour solution* is generated by swapping a visited and an unvisited customer, or by removing one customer and inserting another.
- If the neighbour is better, accept it always. If it is worse (profit decreases by Δ (Delta)), accept it with probability $e^{-\Delta/T}$ (e to the minus Delta over T). This is the *Metropolis criterion*.
- As $T \rightarrow 0$ (T approaches zero), the algorithm becomes greedy and converges to a local optimum.

TOP: Heuristic Algorithms III

3. Tabu Search (TS) (Tang and Miller-Hooks, 2005). Tabu Search avoids cycling by maintaining a *tabu list* — a memory of recently made moves that are *forbidden* (tabu) for the next few iterations. The algorithm:

- 1 Start from a feasible solution.
- 2 Generate all neighbours by applying a move (e.g., swap or remove/insert a customer).
- 3 Select the best non-tabu neighbour (or a tabu one if it is better than the best solution found so far — aspiration criterion).
- 4 Update the tabu list and repeat for a fixed number of iterations.

The tabu list prevents the search from returning to recently visited solutions, forcing exploration of new regions.

4. Iterated Local Search (ILS) (Vansteenwegen et al., 2009). Consistently the best performer on standard benchmarks. Each iteration:

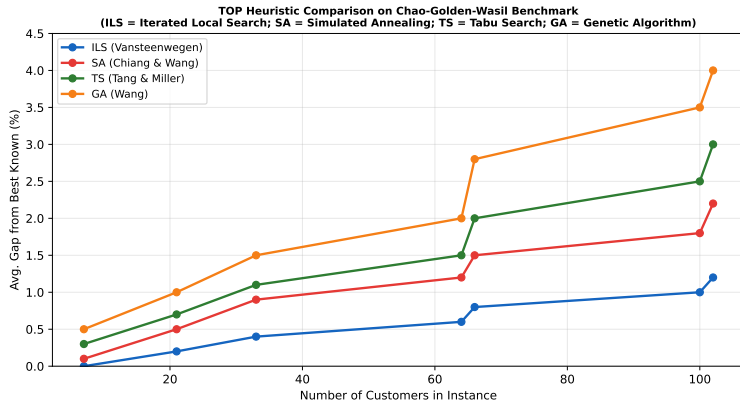
- 1 Apply a **perturbation**: randomly remove r customers from the routes and attempt to re-insert them (possibly from the unvisited pool) in a random order.

TOP: Heuristic Algorithms IV

- ② Apply **local search**: 2-opt on each route, plus *Or-opt* (moving a chain of 1–3 customers to a better position) and customer swap.
- ③ If the new solution improves on the best found, update the best solution.
- ④ Repeat for a budget of iterations.

5. GRASP (Archetti et al., 2007). Archetti, Hertz, and Speranza applied GRASP to the TOP. Each iteration builds a solution by a randomised greedy procedure, then applies local search with 2-opt, Or-opt, and swap moves. They showed that combining construction diversity with local search improvement consistently outperforms pure construction heuristics.

TOP: Heuristic Algorithms VI



TOP: Heuristic Algorithms VII

Figure: Comparison of TOP heuristics on Chao-Golden-Wasil benchmark instances (schematic). The ILS of Vansteenwegen et al. achieves the smallest gap from the best known solution across all instance sizes. Gap is reported as the percentage difference between the heuristic's profit and the best known profit.

TOP: Benchmark Instances (Table 10.2) I

Chao, Golden, and Wasil (1996) created the standard benchmark dataset for the TOP, consisting of 7 sets of instances with 32–288 customers and 2–4 vehicles. The table below shows average % deviation from the best known solution (BKS) for leading algorithms.

Algorithm	Type	Avg. Gap (%)	Time	Reference
Chao, Golden, Wasil	Greedy	8.5	<1s	1996
Chiang & Wang	SA	3.5	5–15s	1996
Tang & Miller	TS	2.5	10–30s	2005
Archetti et al.	GRASP	1.8	5–20s	2007
Vansteenwegen et al.	ILS	0.5	1–5s	2009
Boussier et al.	B&P (exact)	0.0	mins	2007

In the table, “SA” stands for Simulated Annealing, “TS” for Tabu Search, “GRASP” for Greedy Randomised Adaptive Search Procedure, “ILS” for Iterated Local Search, and “B&P” for Branch-and-Price (an exact method). The ILS achieves near-optimal quality (0.5% gap) orders of magnitude faster than the exact method.

Capacitated TOP (CTOP) I

The Capacitated Team Orienteering Problem (CTOP) adds a **vehicle capacity constraint** to the TOP. Each customer i has a *demand* $q_i \geq 0$ (q sub i), representing the amount of goods or service required. Each vehicle can carry at most Q units of load.

The CTOP adds the following constraint to the TOP formulation:

Capacity Constraint

$$\sum_{i \in V \setminus \{0\}} q_i y_{ik} \leq Q \quad \forall k \in \{1, \dots, m\} \quad (\text{CTOP.cap})$$

Here q_i (q sub i) is the demand of customer i , y_{ik} (y sub i k) is 1 if vehicle k visits customer i , and Q (Q) is the vehicle capacity (maximum total load per vehicle). The sum runs over all customers served by vehicle k .

The CTOP thus has two interacting constraints: the time limit $T_{\max}$ and the capacity Q . A vehicle may have remaining time but a full load, or spare capacity but no time left. Optimally balancing these two resources makes the CTOP harder than the TOP.

Capacitated TOP (CTOP) II

Applications. The CTOP models situations such as:

- A delivery truck with limited cargo space visiting customers to maximise revenue, subject to both a distance limit and a weight limit.
- A patrol vehicle collecting evidence from crime scenes, limited by both time and the amount of evidence it can carry.

Boussier, Feillet, and Gendreau (2007) developed the first exact Branch-and-Price algorithm for the CTOP using a column generation framework where each column represents a feasible single-vehicle route.

Variants of VRPPs I

The basic OP and TOP have been extended in many directions to capture additional real-world constraints. This section surveys the most studied variants.

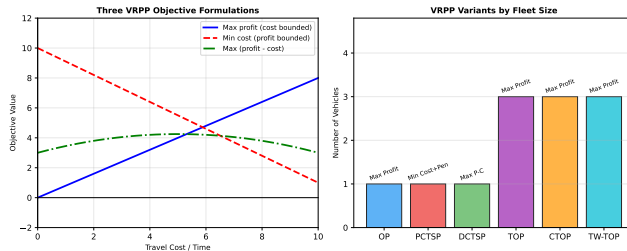
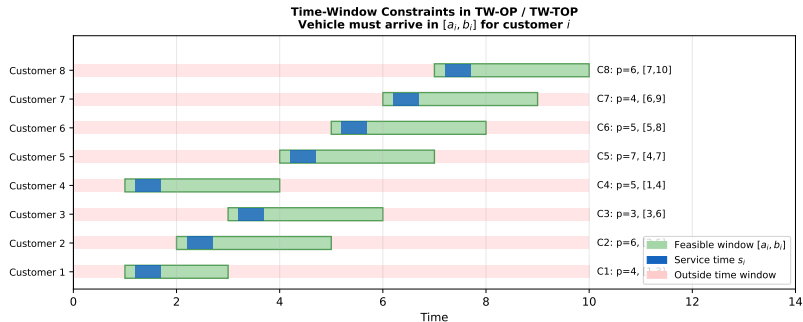


Figure: Left: the three VRPP objective types as functions of travel cost. Right: number of vehicles for each main problem variant.

Time-Window OP and TOP (TW-OP, TW-TOP) I

In the Time-Window OP (TW-OP) and Time-Window TOP (TW-TOP), each customer i has a **time window** $[a_i, b_i]$ (a sub i to b sub i): the vehicle must arrive at customer i no earlier than a_i and no later than b_i . Service at i takes s_i (s sub i) time units.



Time-Window OP and TOP (TW-OP, TW-TOP) II

Figure: Time windows for 8 customers. Green bars = feasible arrival interval $[a_i, b_i]$. Blue bars = service time s_i . Red zones = periods when the vehicle must not arrive. The vehicle's schedule must fit all selected customers' windows simultaneously.

Time-Window OP and TOP (TW-OP, TW-TOP) III

Additional constraint for TW-OP / TW-TOP

Let t_i (t sub i) be the arrival time of the vehicle at customer i . The time-window constraint requires:

$$a_i \leq t_i \leq b_i \quad \forall i \in V \quad (\text{TW})$$

If the vehicle arrives before a_i , it must wait (incurring idle time). If it arrives after b_i , the customer cannot be served and must be skipped.

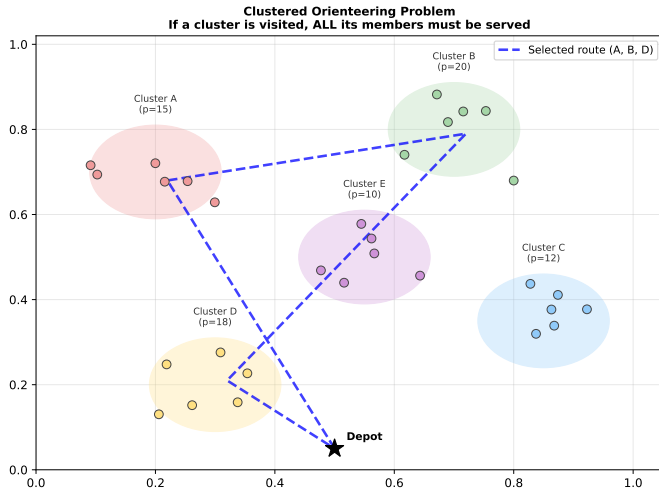
Why time windows matter. Time windows appear in home delivery (customers are home only during certain hours), medical logistics (blood samples must arrive at the lab before they expire), and tourism (attractions are open only at certain times).

Methods for TW-OP / TW-TOP. Vansteenwegen, Souffriau, Vanden Berghe, and Van Oudheusden (2009) proposed an ILS that was extended to handle time windows. The time-window feasibility check is incorporated into the route evaluation procedure. Scheduling constraints interact with customer selection: a customer that is profitable may still be skipped because its time window forces an infeasible detour.

Clustered Orienteering Problem I

In the Clustered Orienteering Problem (COP), customers are grouped into **clusters**. The routing rule is: *if any customer from a cluster is visited, then all customers in that cluster must be visited*. Equivalently, the vehicle either services a cluster completely or skips it entirely.

Clustered Orienteering Problem II



Clustered Orienteering Problem III

Figure: Clustered OP: five clusters (coloured circles), each containing six customers. The selected route visits clusters A, B, and D entirely, collecting all their members' profits. Clusters C and E are completely skipped.

Clustered Orienteering Problem IV

COP formulation addition

Let C_ℓ (C sub ℓ) denote the set of customers in cluster ℓ . The cluster-visit constraint is:

$$y_i = y_j \quad \forall i, j \in C_\ell, \forall \ell \quad (\text{COP.cluster})$$

All customers in the same cluster must have the same visit status: either all visited ($y_i = 1$ for all $i \in C_\ell$) or all skipped ($y_i = 0$ for all $i \in C_\ell$).

Applications. The COP models tourist itinerary planning where a tourist must see all attractions of a museum (a cluster) if she enters it, or she skips the museum entirely. Similarly, a service engineer must complete all maintenance tasks at a factory site (cluster) before leaving.

Algorithms. Angelelli, Archetti, and Vindigni (2014) developed a branch-and-cut algorithm for the COP. The key algorithmic insight is to treat each cluster as a single super-node in the planning phase, then expand it to its component customers once the cluster is selected.

Multi-Period Team Orienteering Problem I

In the Multi-Period TOP (also called the Selective Periodic Vehicle Routing Problem, SPVRP), planning is done over a **horizon of τ (tau) periods** (e.g., $\tau = 5$ working days).

- The same fleet of m vehicles is available each period.
- Each customer i has a **visit frequency** f_i ($f_{\text{sub } i}$): the minimum number of periods in which customer i must be visited over the planning horizon.
- The objective is to maximise total profit across all periods, subject to per-period time limits and visit frequency requirements.

Multi-Period Team Orienteering Problem II

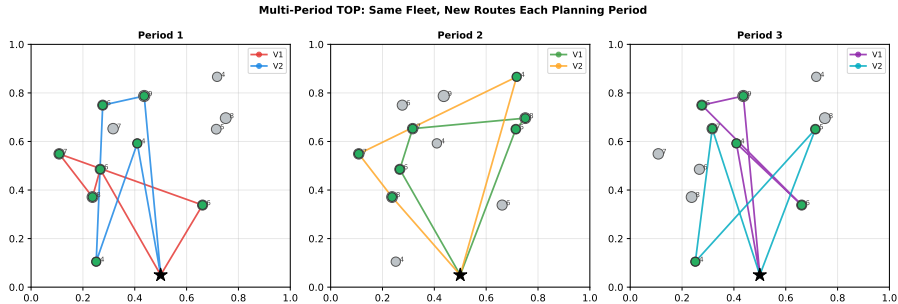


Figure: Multi-period TOP over 3 planning periods. Green nodes = visited in that period; grey = not visited. Different customers may be served in different periods; the fleet reconfigures each day.

Multi-Period Team Orienteering Problem III

Multi-Period TOP – Key Constraint

Let $y_{it} \in \{0, 1\}$ (y sub i t) be 1 if customer i is visited in period t . The visit frequency constraint requires:

$$\sum_{t=1}^{\tau} y_{it} \geq f_i \quad \forall i \in V \setminus \{0\} \quad (\text{MP.freq})$$

Some customers must be visited in at least f_i of the τ (tau, pronounced “taw”) periods. For customers with $f_i = 0$, visiting is optional.

Applications. Multi-period TOP models periodic service delivery (e.g., waste collection or meter reading where each customer must be visited with a given frequency), periodic inspection routes, and recurring tourist guide services.

Methods. Archetti, Bianchessi, and Speranza (2014) developed a branch-and-price exact algorithm, decomposing the problem by period. Heuristic approaches include ILS with period-aware perturbation and genetic algorithms that evolve multi-period schedules.

VRP with Private Fleet and Common Carrier (SPS) I

A particularly practical variant is the **Selective and Periodic Vehicle Routing Problem (SPS)**, which combines:

- 1 A **private fleet** (owned vehicles, fixed cost per period, limited capacity and time).
- 2 A **common carrier** (outsourcing option): any customer not served by the private fleet can be outsourced at a per-customer cost.

The decision maker must choose which customers to serve with the private fleet (cheaper on a per-delivery basis) and which to outsource to the common carrier (more expensive but removes the routing burden).

SPS Objective

Minimise

$$\sum_{k=1}^m \text{RouteCost}(R_k) + \sum_{i \notin \text{served}} \text{OutsourceCost}(i)$$

where $\text{RouteCost}(R_k)$ is the total travel cost of vehicle k 's route R_k , and $\text{OutsourceCost}(i)$ is the penalty for outsourcing customer i to the common carrier.

VRP with Private Fleet and Common Carrier (SPS) II

This formulation is equivalent to a PCTSP with multiple vehicles: the outsourcing cost plays the role of the penalty π_i . Archetti, Bianchessi, and Speranza (2014) showed that exact and heuristic methods for the PCTSP can be adapted to the SPS by decomposing the multi-vehicle problem into single-vehicle subproblems.

Python: Greedy OP Solver

The following Python code implements a greedy profit-per-distance ratio heuristic for the Orienteering Problem. The function takes a list of customer coordinates, their profits, the depot location, and the time limit $T_{\max}$, and returns the best route found.

```
import numpy as np

def euclidean(p1, p2):
    """Euclidean distance between two 2D points."""
    return np.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def route_cost(route, depot, customers):
    """Total travel cost of route (depot -> c1 -> ... -> ck -> depot)."""
    nodes = [depot] + [customers[i] for i in route] + [depot]
    return sum(euclidean(nodes[i], nodes[i+1]) for i in range(len(nodes)-1))

def greedy_op(depot, customers, profits, T_max):
    """
    Greedy OP heuristic based on profit/distance ratio.
    Returns list of customer indices in the selected route.
    """
    n = len(customers)
    # Compute profit-to-depot-distance ratio for each customer
    ratios = [profits[i] / (euclidean(depot, customers[i]) + 1e-9)
               for i in range(n)]
    # Sort by ratio (descending)
    order = sorted(range(n), key=lambda i: -ratios[i])
```

Python: 2-opt Local Search for OP Route Improvement

Once a route is constructed, its quality can be improved by **2-opt**. The 2-opt move removes two edges from the route and reconnects the two resulting segments in the only other possible way (reversing the segment between the removed edges). If the reversal reduces total cost, the move is accepted. This process repeats until no improving 2-opt move exists (a *local optimum*).

```
def two_opt(route, depot, customers, T_max):
    """
    Apply 2-opt improvement to a given route.
    Returns the improved route (list of customer indices).
    """
    improved = True
    best_route = route[:]
    while improved:
        improved = False
        best_cost = route_cost(best_route, depot, customers)
        for i in range(len(best_route) - 1):
            for j in range(i + 2, len(best_route)):
                # Reverse the segment between positions i+1 and j (inclusive)
                new_route = (best_route[:i+1]
                             + best_route[i+1:j+1][::-1]
                             + best_route[j+1:])
                new_cost = route_cost(new_route, depot, customers)
                if new_cost < best_cost - 1e-9 and new_cost <= T_max:
                    best_route = new_route
                    best_cost = new_cost
                    improved = True
```

Python: Simple TOP Solver (Multi-Vehicle Greedy)

This code extends the greedy OP solver to the Team Orienteering Problem by allocating customers greedily across multiple vehicles.

```
def greedy_top(depot, customers, profits, T_max, m):  
    """  
    Greedy TOP heuristic: build m routes greedily.  
    Returns list of m routes (each route is a list of customer indices).  
    """  
  
    n = len(customers)  
    visited = set()  
    routes = [[] for _ in range(m)]  
  
    # Compute ratios for all customers  
    ratios = [profits[i] / (euclidean(depot, customers[i]) + 1e-9)  
              for i in range(n)]  
  
    # Each vehicle builds its route greedily  
    for k in range(m):  
        improved = True  
        while improved:  
            improved = False  
            best_gain, best_i, best_pos = 0, None, None  
            current_cost = route_cost(routes[k], depot, customers)  
  
            for i in range(n):  
                if i in visited:  
                    continue # already assigned to another vehicle
```

Conclusions I

This chapter surveyed the Vehicle Routing Problems with Profits (VRPPs), a rich family of combinatorial optimisation problems where visiting a customer is a decision variable, not a fixed requirement.

The key take-aways are:

- ➊ **Customer selection is the defining challenge.** Unlike classical VRP, in VRPPs the algorithm must simultaneously decide *who* to visit and *in what order* — a harder combinatorial problem.
- ➋ **Three objective types structure the family.** Maximise profit subject to a time limit (OP/TOP), minimise cost subject to a minimum profit (PCTSP), or maximise net profit with no hard constraint (PTP/DCTSP) — each models a different operational context.
- ➌ **Exact methods are limited to small/medium instances.** Branch-and-cut (Fischetti et al.), branch-and-price (Boussier et al.), and ILP solvers can handle instances up to about $n = 100$ customers optimally. For larger instances, heuristics are necessary.
- ➍ **ILS and GRASP are the most effective heuristics for TOP.** Vansteenwegen et al.'s ILS achieves an average gap of less than 0.5% from the best known solution on standard benchmarks, in seconds of computing time.

- ⑤ **Variants are abundant and practically important.** Time windows, cluster constraints, multi-period planning, and capacity constraints add realism and difficulty. Each variant requires tailored modelling and algorithmic adaptations.

The Central Trade-off in VRPPs

Every VRPP solution must balance two opposing forces: (1) the desire to visit more customers to collect more profit, and (2) the constraint (time, capacity, or cost) that limits how many customers can be visited. Understanding and exploiting this trade-off — analytically through the mathematical model, and computationally through smart heuristics — is the core intellectual challenge of the field.

Future Research Directions I

Despite decades of research, many open problems and new directions remain active.

- 1. Stochastic VRPPs.** In practice, profits and travel times are uncertain. Stochastic OP and TOP models (where p_i is a random variable) require robust or chance-constrained optimisation. The vehicle should visit customers whose *expected* profit is high and whose variance is low (risk-averse routing).
- 2. Dynamic VRPPs.** Customers may appear or cancel during the planning horizon. Real-time rerouting methods (online algorithms) must make decisions without full information about future customers.
- 3. Multi-objective VRPPs.** Decision makers often have multiple competing objectives (profit, environmental cost, customer satisfaction). Pareto-frontier methods and evolutionary multi-objective algorithms are under active development.
- 4. Large-scale instances.** Current exact methods are limited to $n \approx 100$. Machine learning-guided branch-and-bound, learning-to-search approaches, and neural combinatorial optimisation offer promising new directions for solving instances with $n > 1,000$ customers.

Future Research Directions II

5. UAV and multi-modal routing. Unmanned aerial vehicles (UAVs / drones) operate under energy budgets analogous to time limits in the OP. Multi-modal VRPPs (where a truck deploys drones that return with collected data) are a rapidly growing application area.

6. Connections to other problems. The PCTSP is a special case of the Capacitated Vehicle Routing Problem (CVRP) with an infinite vehicle; understanding these connections facilitates algorithm transfer and unified solver frameworks.

Summary of All Mathematical Models I

For easy reference, we collect all objective functions and their distinguishing constraints.

Problem	Objective (maximise/minimise)	Distinguishing constraint
OP	$\max \sum_i p_i y_i$	$\sum_{(i,j)} c_{ij} x_{ij} \leq T_{\max}$
PCTSP	$\min \sum_{(i,j)} c_{ij} x_{ij} + \sum_i \pi_i (1 - y_i)$	$\sum_i p_i y_i \geq P_{\min}$
PTP	$\max \sum_i p_i y_i - \sum_{(i,j)} c_{ij} x_{ij}$	None (unconstrained)
TOP	$\max \sum_k \sum_i p_i y_{ik}$	$\sum_{(i,j)} c_{ij} x_{ijk} \leq T_{\max} \quad \forall k; \sum_k y_{ik} \leq 1 \quad \forall i$
CTOP	Same as TOP	TOP constraints + $\sum_i q_i y_{ik} \leq Q \quad \forall k$
TW-OP/TOP	Same as OP/TOP	Time windows: $a_i \leq t_i \leq b_i \quad \forall i$

Summary of All Mathematical Models II

Symbol legend: p_i = profit of customer i ; c_{ij} = travel cost from i to j ; x_{ij} (x_{ijk}) = arc use variable (vehicle k specific); y_i (y_{ik}) = customer visit variable; π_i (pi sub i) = penalty for skipping customer i ; $T_{\max}$ = time budget per route; $P_{\min}$ = minimum profit requirement; q_i = demand of customer i ; Q = vehicle capacity; a_i , b_i = time-window bounds for customer i ; m = number of vehicles; k = vehicle index.

Key References I

The following references are cited in this chapter and cover both the foundational papers and the most influential algorithmic contributions.

- **Tsiligirides (1984)**. “Heuristic methods applied to orienteering.” *Journal of the OR Society*, 35, 797–809. *First paper to define and study the OP.*
- **Golden, Levy, Vohra (1987)**. “The orienteering problem.” *Naval Research Logistics*, 34, 307–318. *Formal definition and first systematic study.*
- **Balas (1989)**. “The prize-collecting traveling salesman problem.” *Networks*, 19, 621–636. *Introduced PCTSP; motivated by rolling mill scheduling.*
- **Chao, Golden, Wasil (1996)**. “The team orienteering problem.” *European Journal of OR*, 88, 464–474. *Defined TOP; provided first heuristic and benchmark.*
- **Fischetti, Salazar, Toth (1998)**. “Solving the orienteering problem through branch-and-cut.” *INFORMS J. on Computing*, 10, 133–148. *First effective exact algorithm for OP.*
- **Chiang, Wang (1996)**. “A knowledge-based system for vehicle routing using Tabu search.” *Computers & OR*, 23, 609–628. *SA heuristic for TOP.*
- **Archetti, Hertz, Speranza (2007)**. “Metaheuristics for the team orienteering problem.” *J. of Heuristics*, 13, 49–76. *GRASP for TOP; systematic computational comparison.*
- **Vansteenwegen, Souffriau, Van Oudheusden (2011)**. “The orienteering problem: a survey.” *European Journal of OR*, 209, 1–10. *Comprehensive survey of OP and variants up to 2011.*

Key References II

- **Boussier, Feillet, Gendreau (2007).** “An exact algorithm for team orienteering problems.” *4OR*, 5, 211–230. *First branch-and-price exact algorithm for TOP.*
- **Archetti, Speranza, Vigo (2014).** “Chapter 10: Vehicle Routing Problems with Profits.” In *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed., SIAM, pp. 273–297. *Source chapter for these slides.*